CS148a Lec 16

Foundation Models, CLIP, DINO

Raphi Kang
Feb 26, 2026

Roadmap

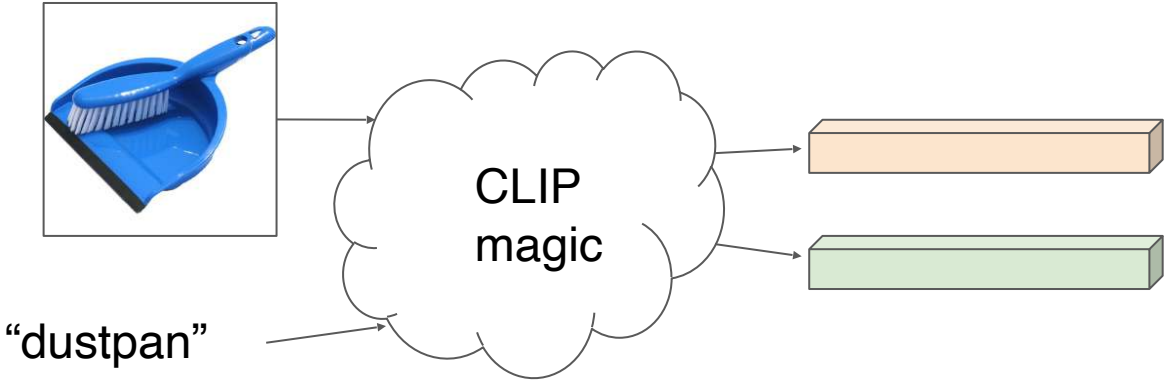
1. Questions about Projects
2. CLIP Motivation
3. How it works
4. Use Cases
5. Limitations
6. DINO overview (quick)

Questions about Projects?

CLIP (Contrastive Language Image Pretraining)

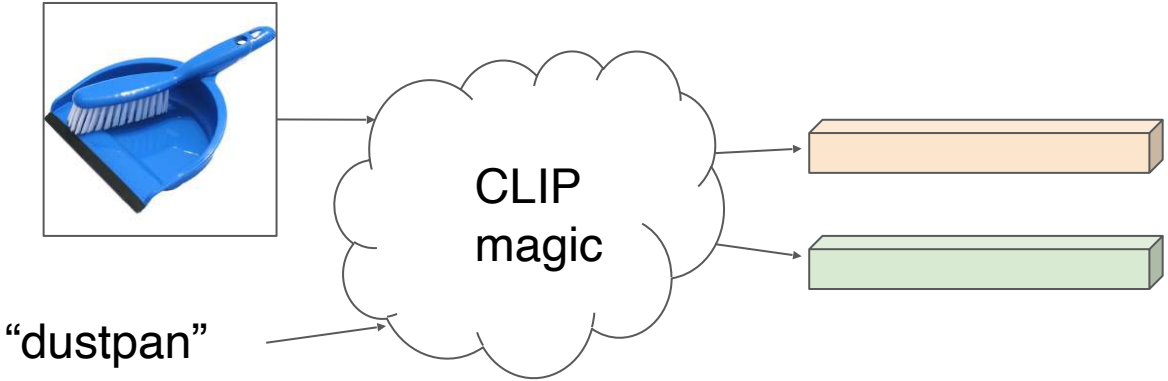
CLIP is a “foundation model” that is *weakly supervised*.

This means that you can put in images and texts, get embeddings, and use it for whatever you want down the line.



CLIP (Contrastive Language Image Pretraining)

The goal of CLIP training is to get a semantically organized *latent space* through *contrastive training.*



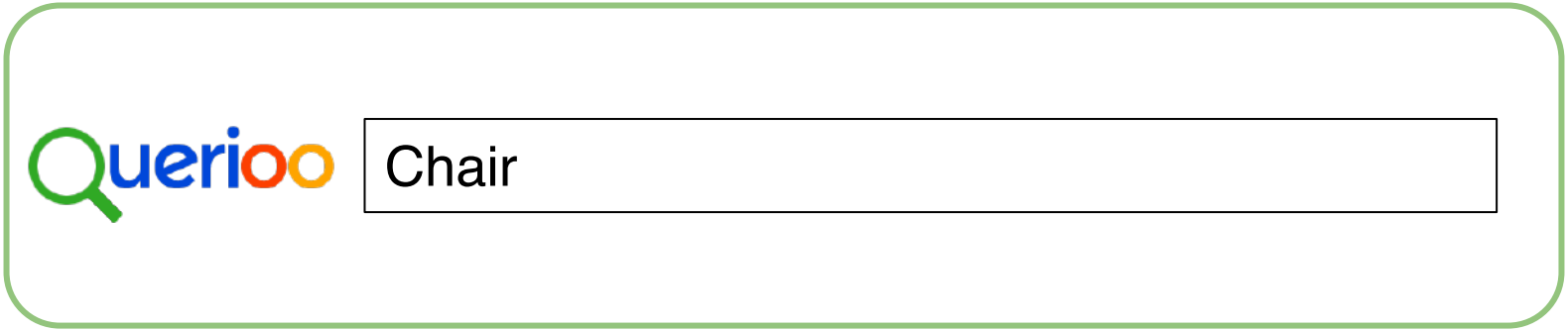
CLIP Motivation

Image Retrieval: say you have this bank of images
(e.g. Amazon, Google Search, Pinterest, etc.)

Image Bank

CLIP Motivation

Image Retrieval: Your goal is to take a customer query and grab the item page for the most relevant thing



CLIP Motivation

Querioo | Chair

If you could get some kind of score between any arbitrary text query and image, you could order your search results

0.2	0.4	0.99	0.99	0.5

CLIP Motivation

Querioo

Blue

1.0

0.89

0.0

CLIP Motivation

Querioo

Blue Dustbin

1.0

0.29

0.0

CLIP Motivation

How do you get these scores?

CLIP Motivation

How do you get these scores?

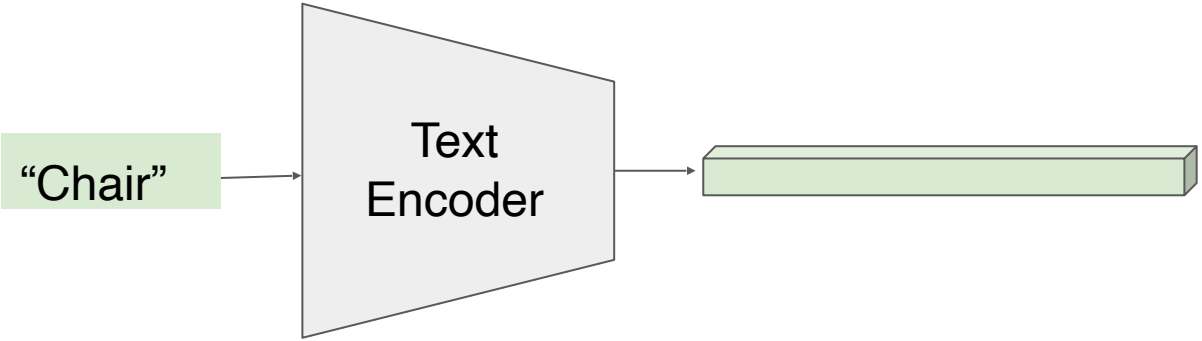
We learned about “embeddings”.
They take plain text → make it into a vector.

CLIP Motivation

How do you get these scores?

We learned about “embeddings”.

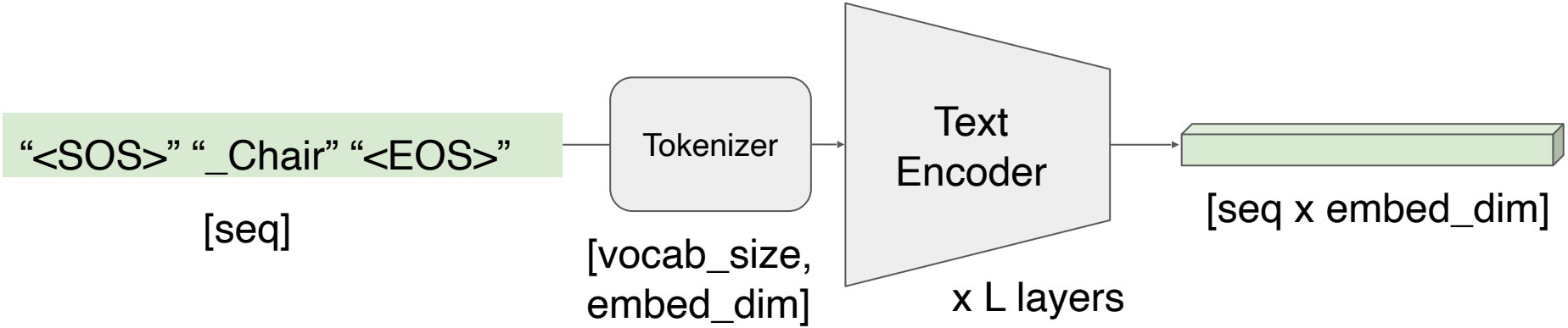
They take plain text → make it into a vector.



CLIP Motivation

How do you get these scores?

We learned about “embeddings”.
They take plain text → make it into a vector.

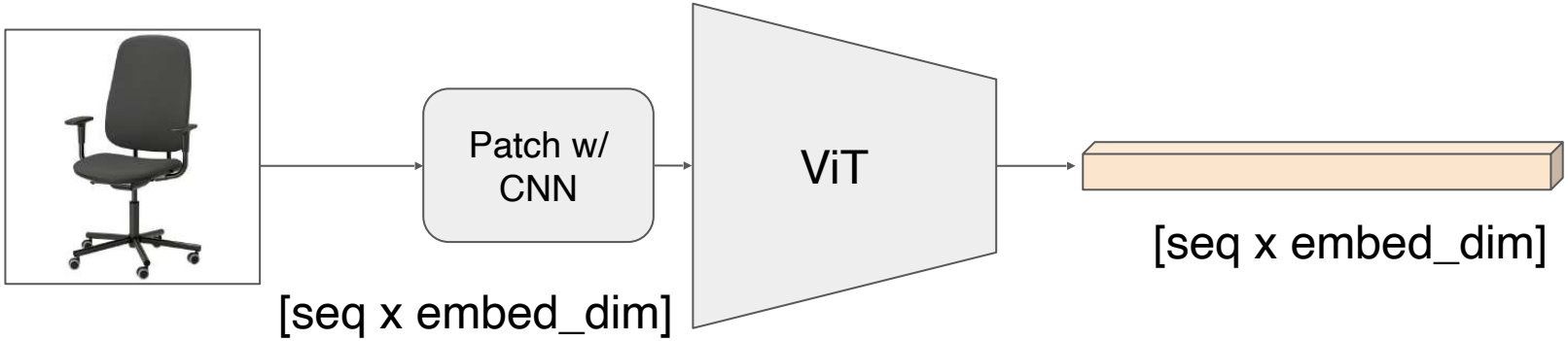


CLIP Motivation

How do you get these scores?

seq = (img_dim// patch_size)**2 + 1

You can also do it for images!



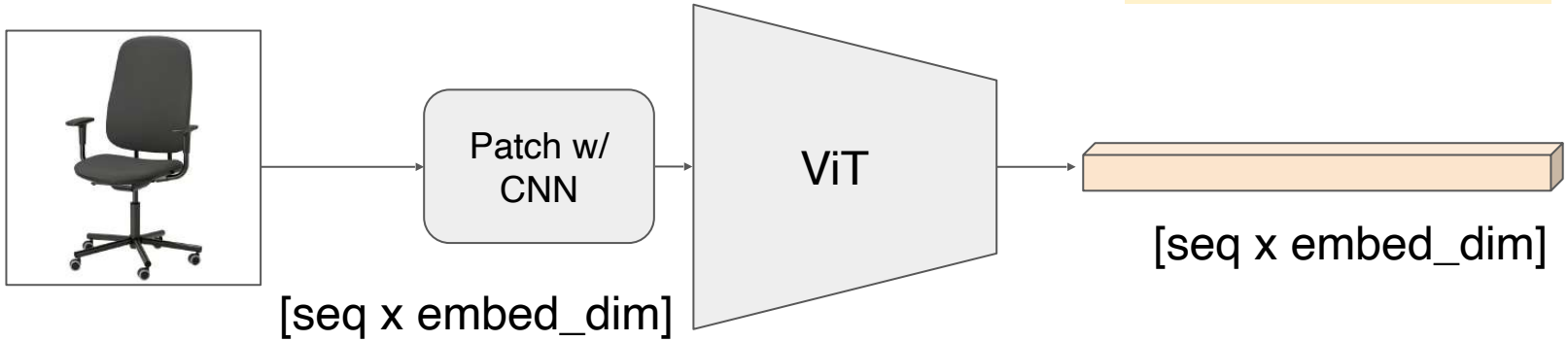
CLIP Motivation

How do you get these scores?

seq = (img_dim// patch_size)**2 + 1

Anyone remember why we have this?

You can also do it for images!

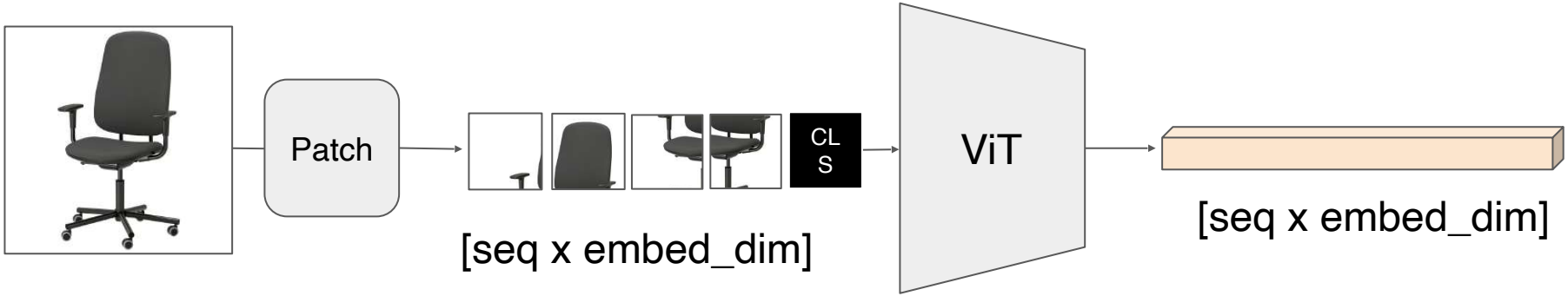


CLIP Motivation

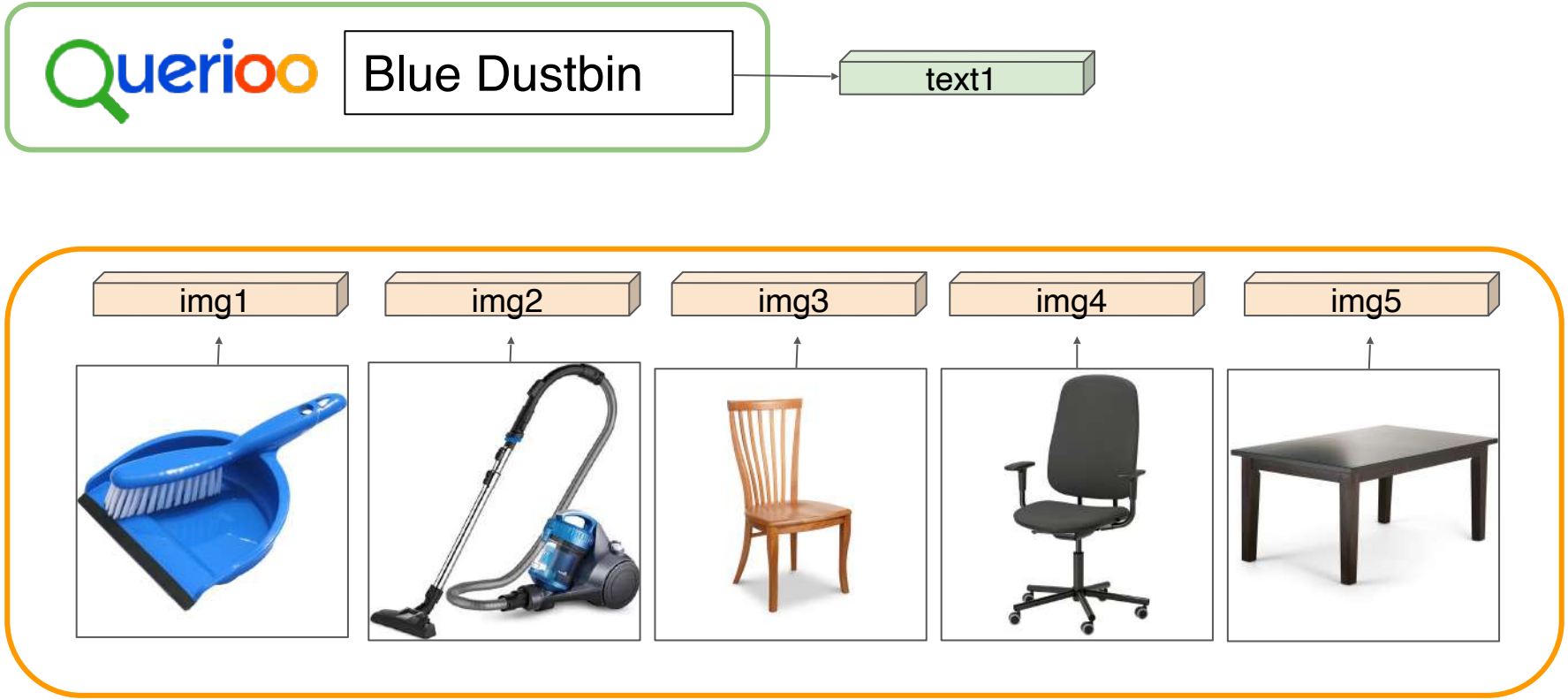
How do you get these scores?

seq = (img_dim// patch_size)**2 + 1

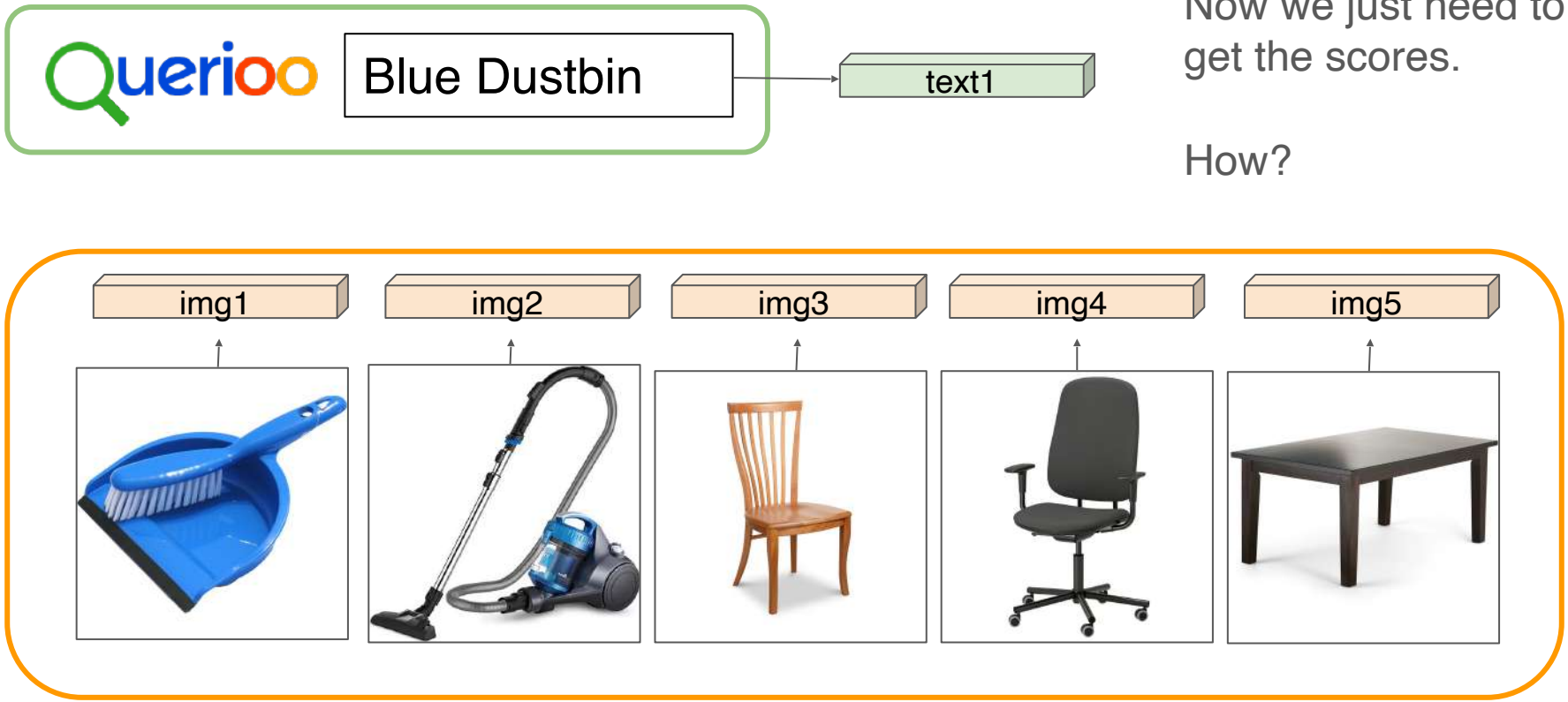
You can also do it for images!



CLIP Motivation



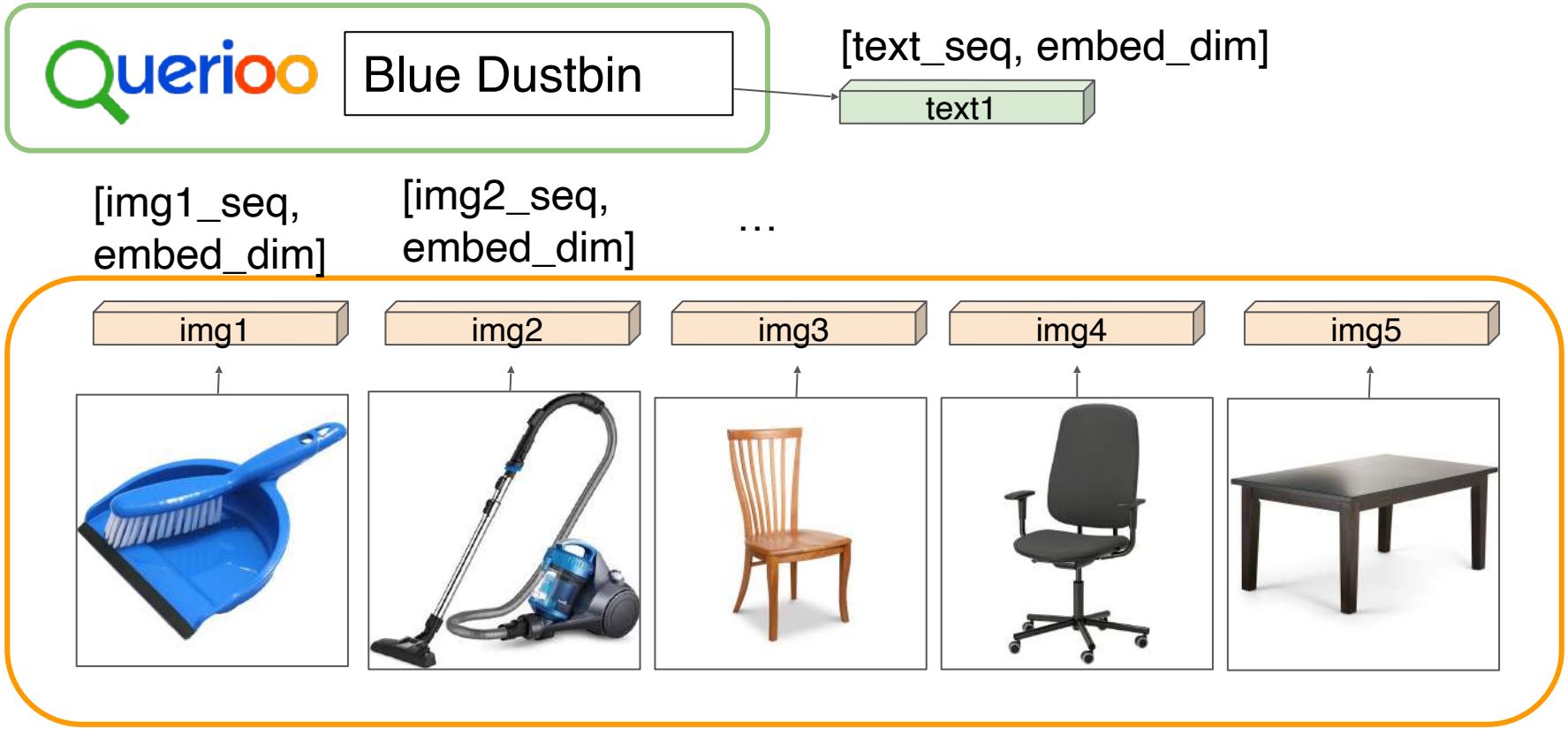
CLIP Motivation



Now we just need to get the scores.

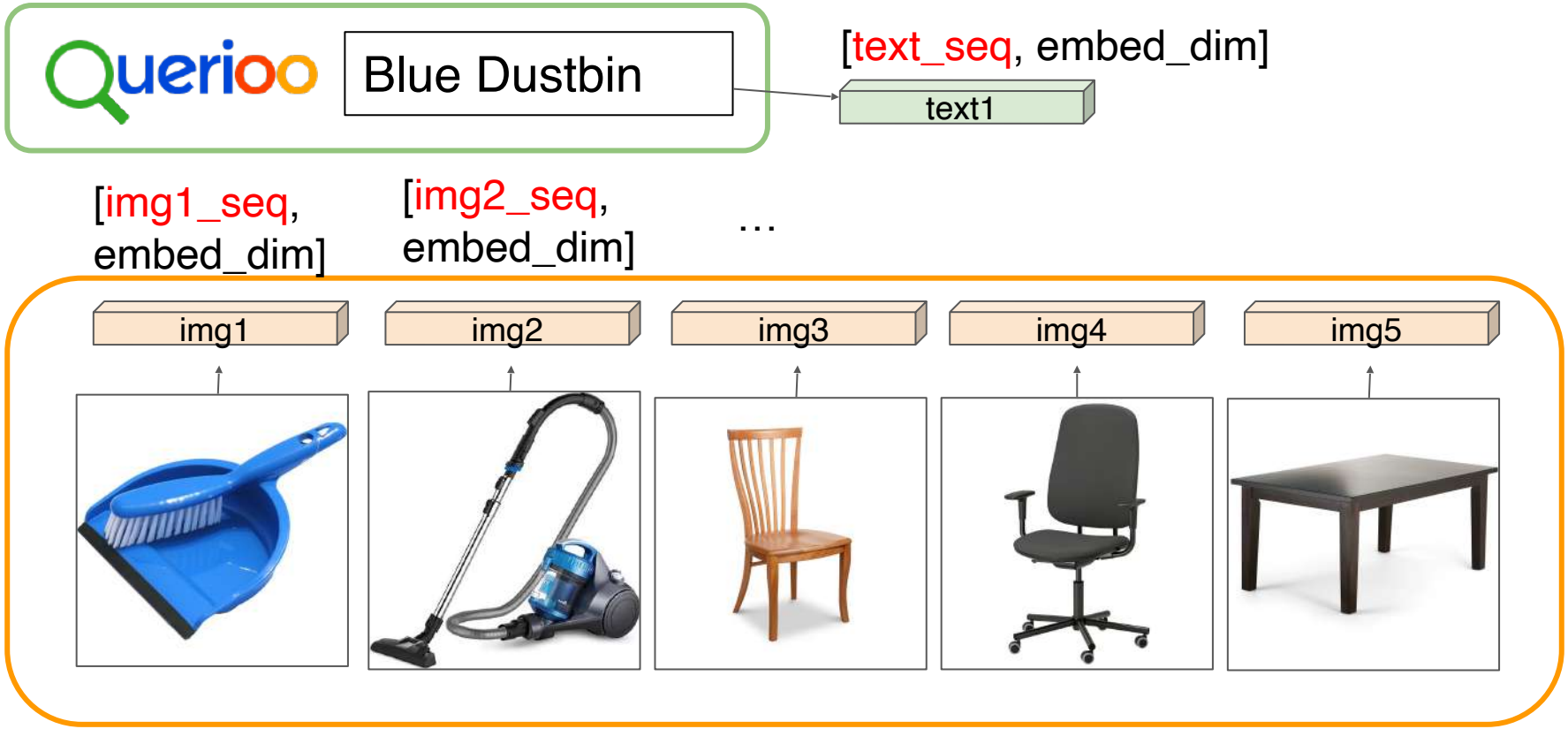
How?

CLIP Motivation



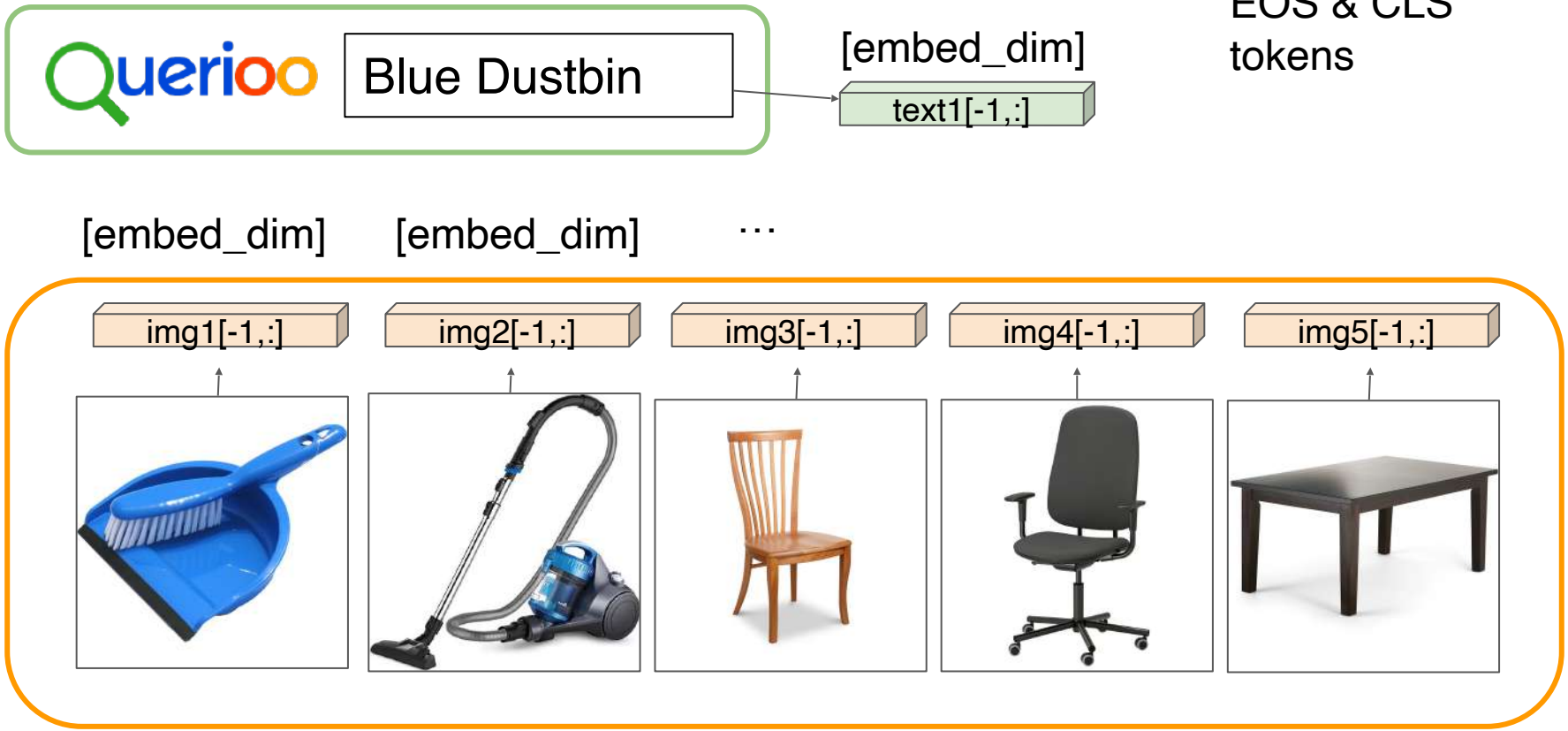
CLIP Motivation

Problem: these numbers are not the same!



CLIP Motivation

💡 Idea: use the EOS & CLS tokens



CLIP Motivation

Querioo

Blue Dustbin

[embed_dim]

text1[-1,:]
/|text1[-1,:]|

💡 Idea: use the EOS & CLS tokens
And normalize the vectors!

[embed_dim] [embed_dim] ...

img1[-1,:]
/|img1[-1,:]|

img2[-1,:]
/ norm

img3[-1,:]
/norm

img4[-1,:]
/norm

img5[-1,:]
/norm

CLIP Motivation

💡 Idea: use the EOS & CLS tokens
And normalize the
tors!

Querioo

Blue Dustbin

[embed_dim]

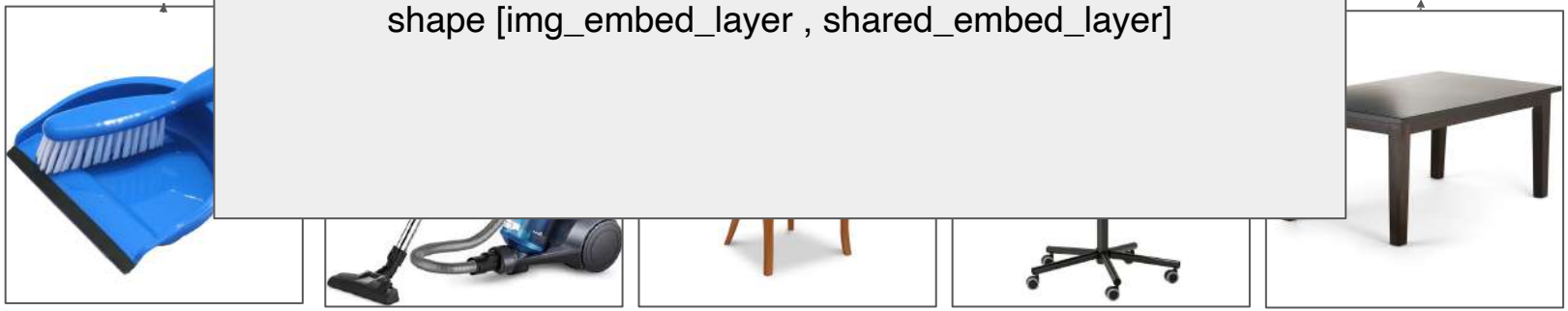
text1[-1,:]

[embed_d

img1[-1,
/limg1[-1,

mg5[-1,:]
/norm

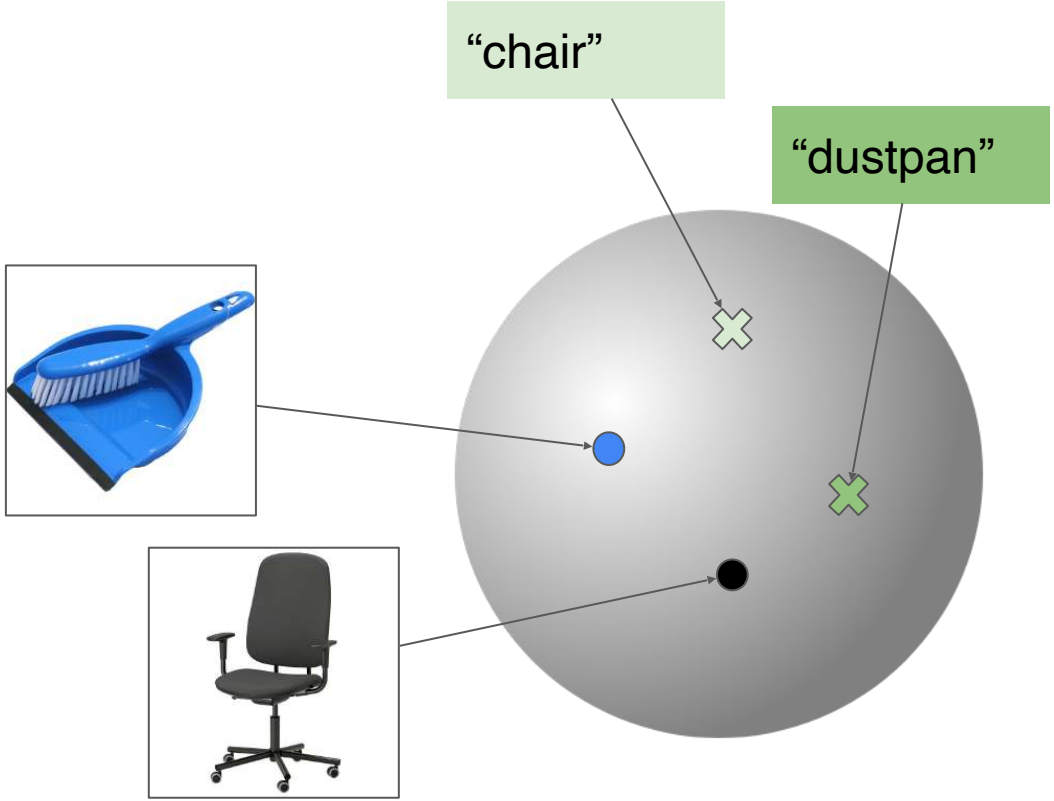
If embed_dim is not the same between image encoder and text encoder – not to worry! We can just add a projection layer that is shape [img_embed_layer , shared_embed_layer]



CLIP Intuition

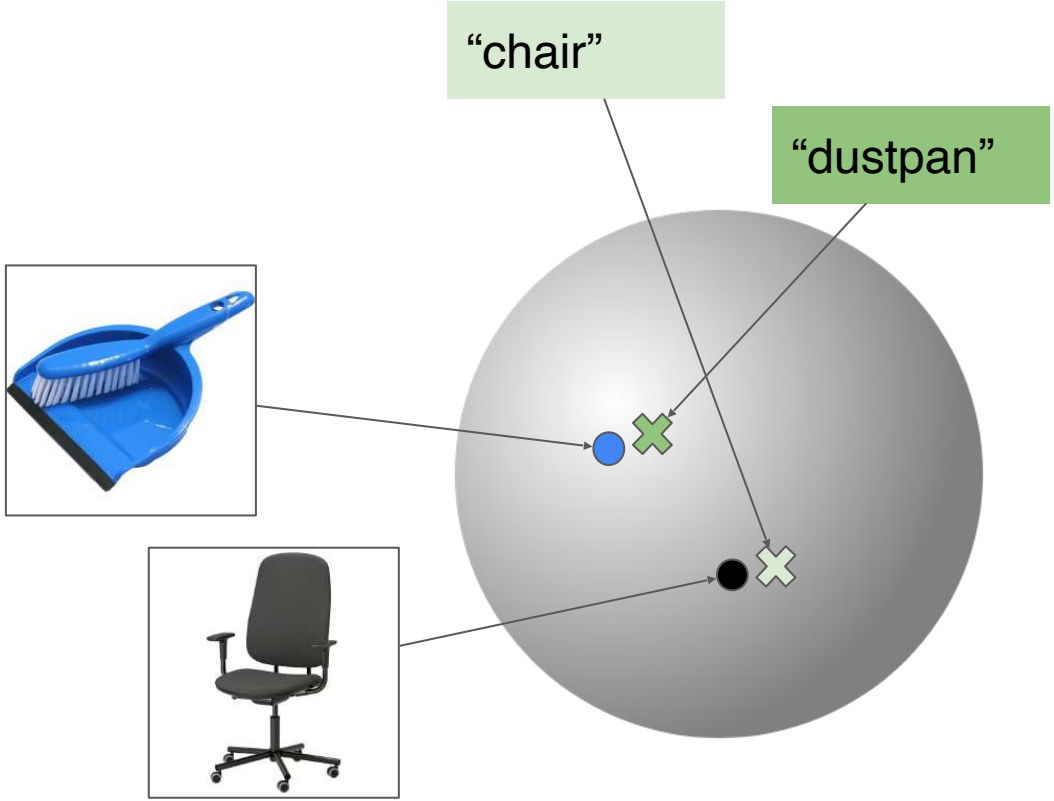
Now we can represent images and texts as vectors on the surface of a unit norm hypersphere in an N-dimensional space,

where N = embed_dim



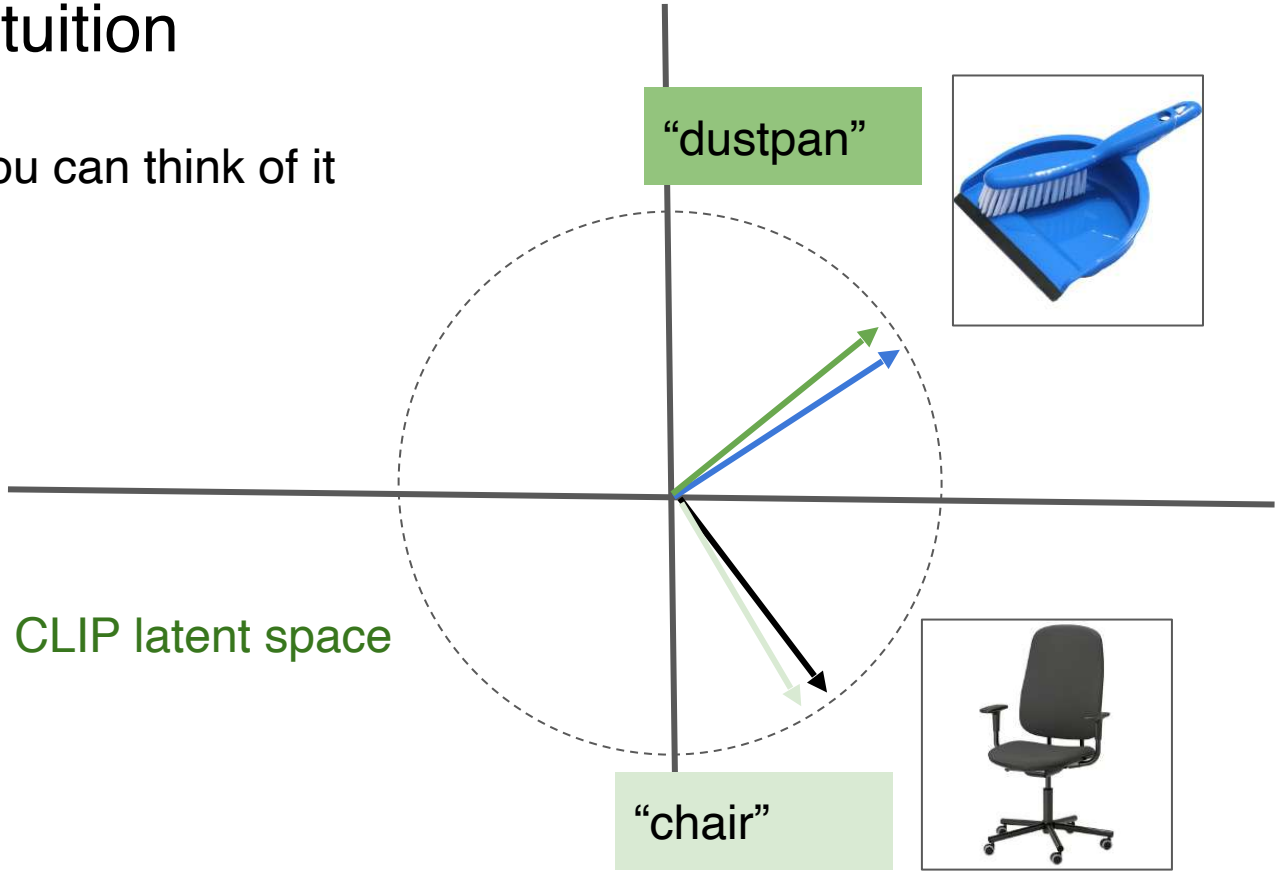
CLIP Intuition

Ideally, after training the vectors would look like this:



CLIP Intuition

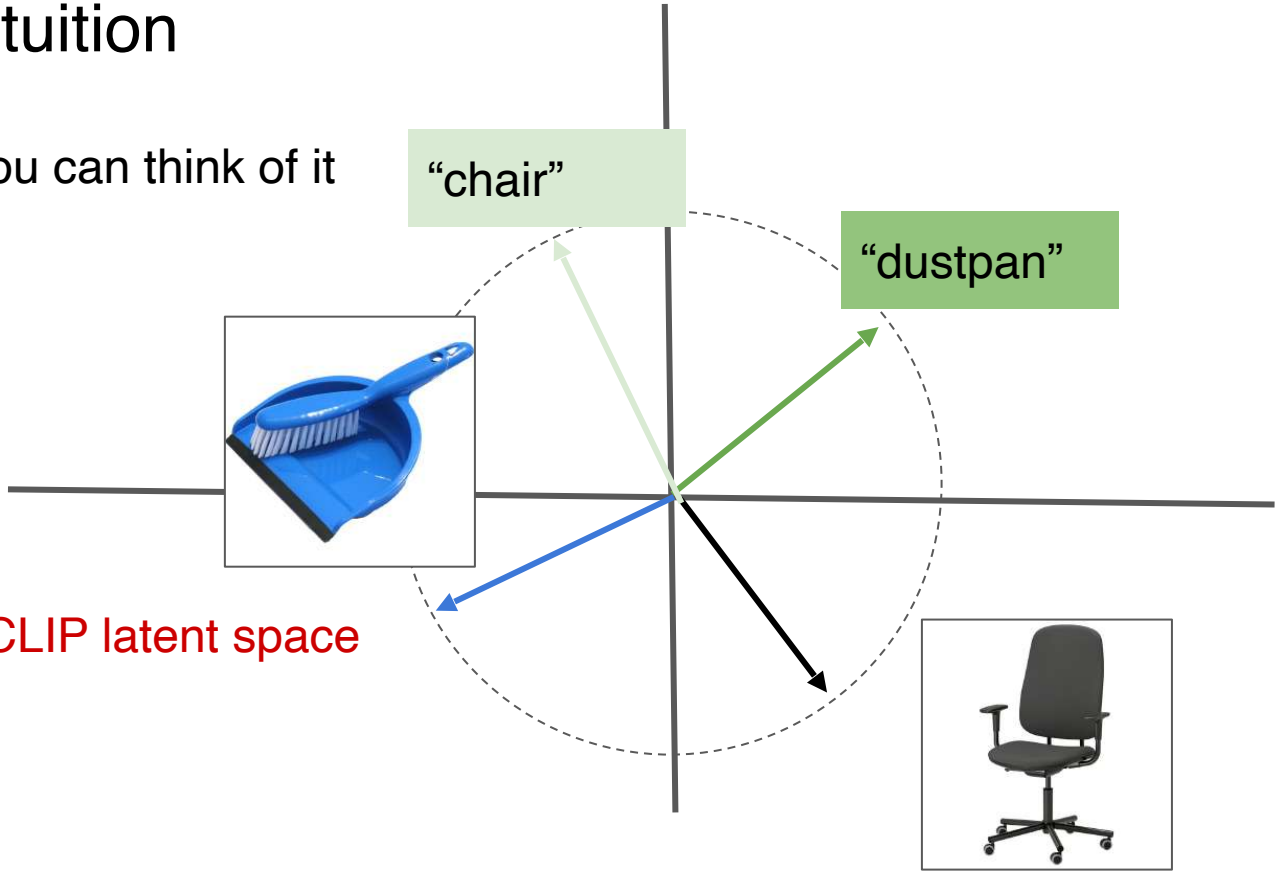
If N = 2, you can think of it like:



Good CLIP latent space

CLIP Intuition

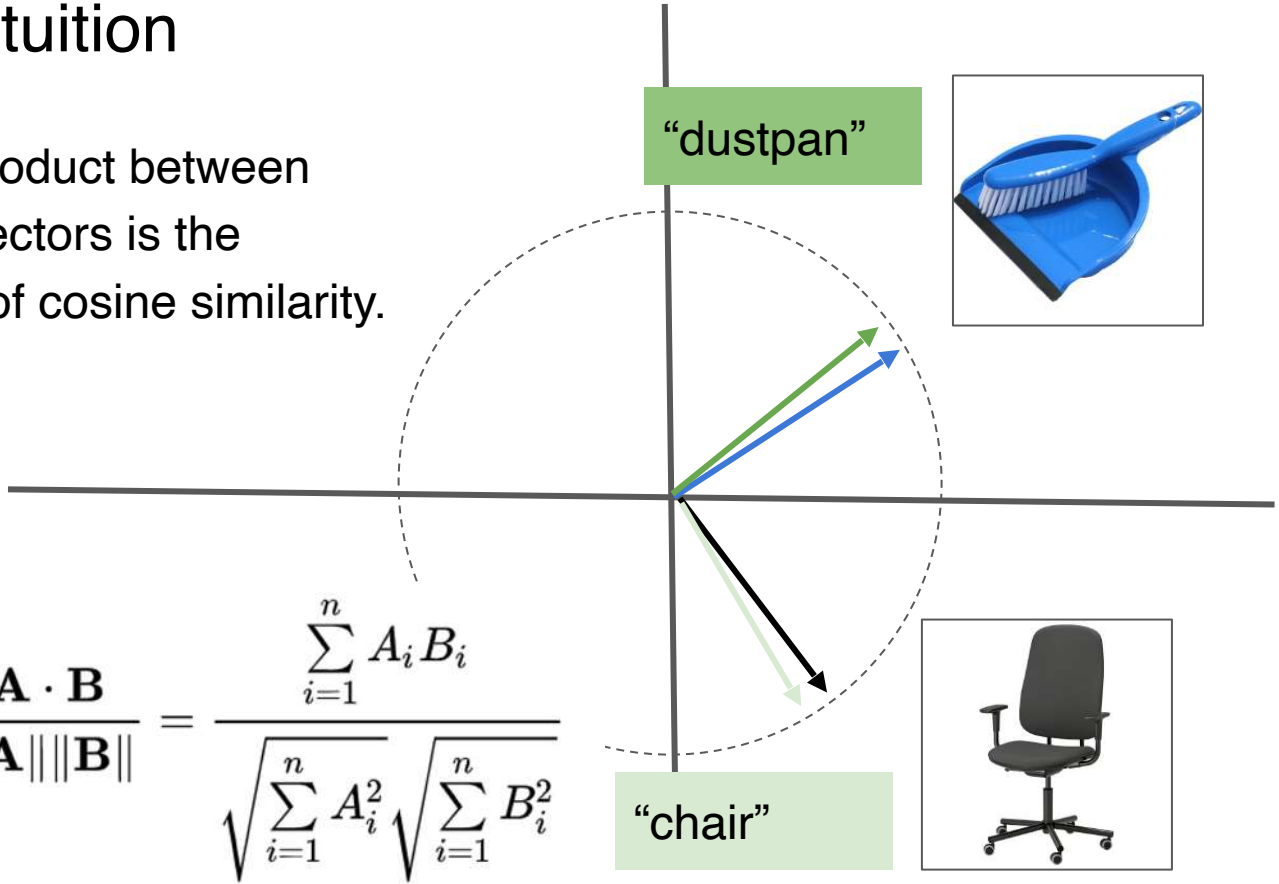
If N = 2, you can think of it like:



Bad CLIP latent space

CLIP Intuition

The dot product between two unit vectors is the definition of cosine similarity.

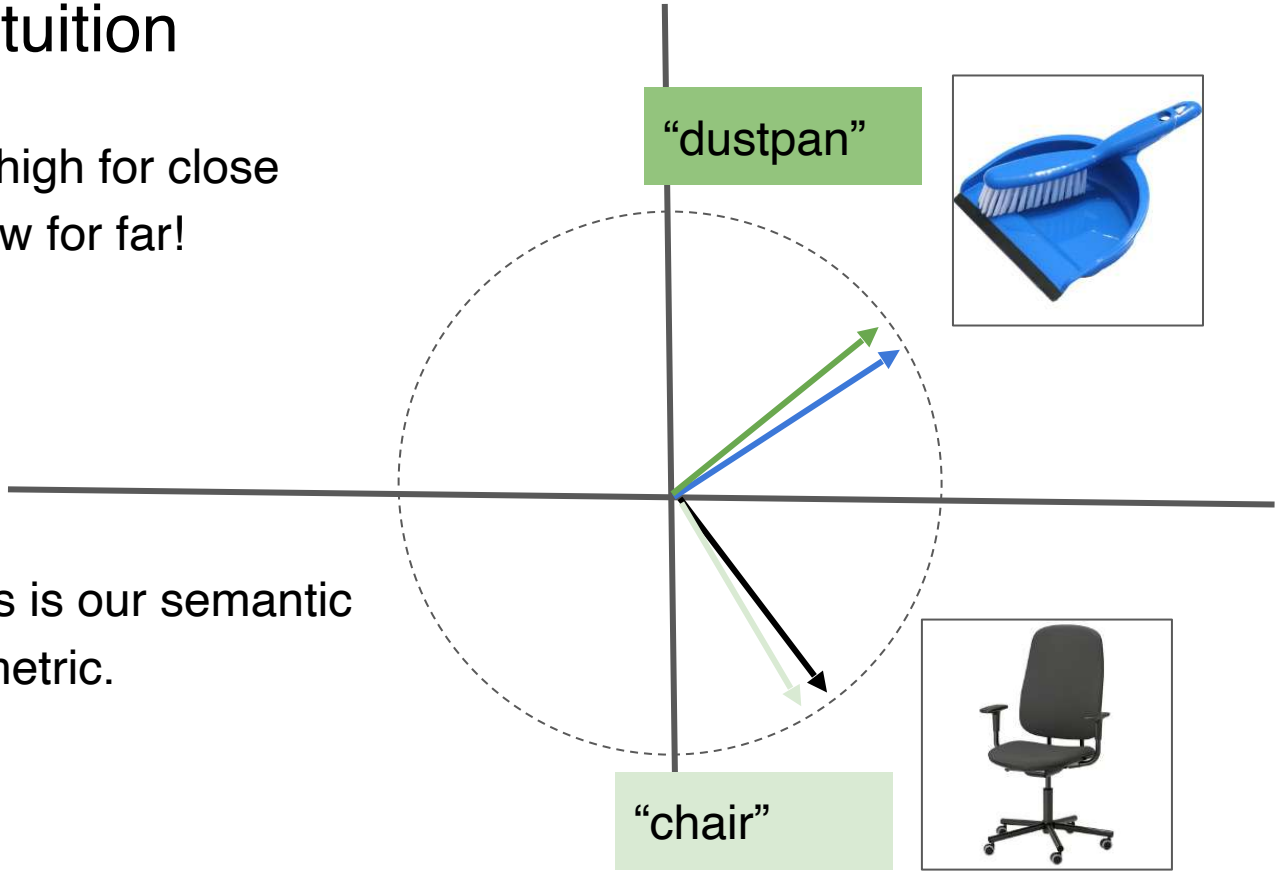


$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

CLIP Intuition

Cossim = high for close vectors, low for far!

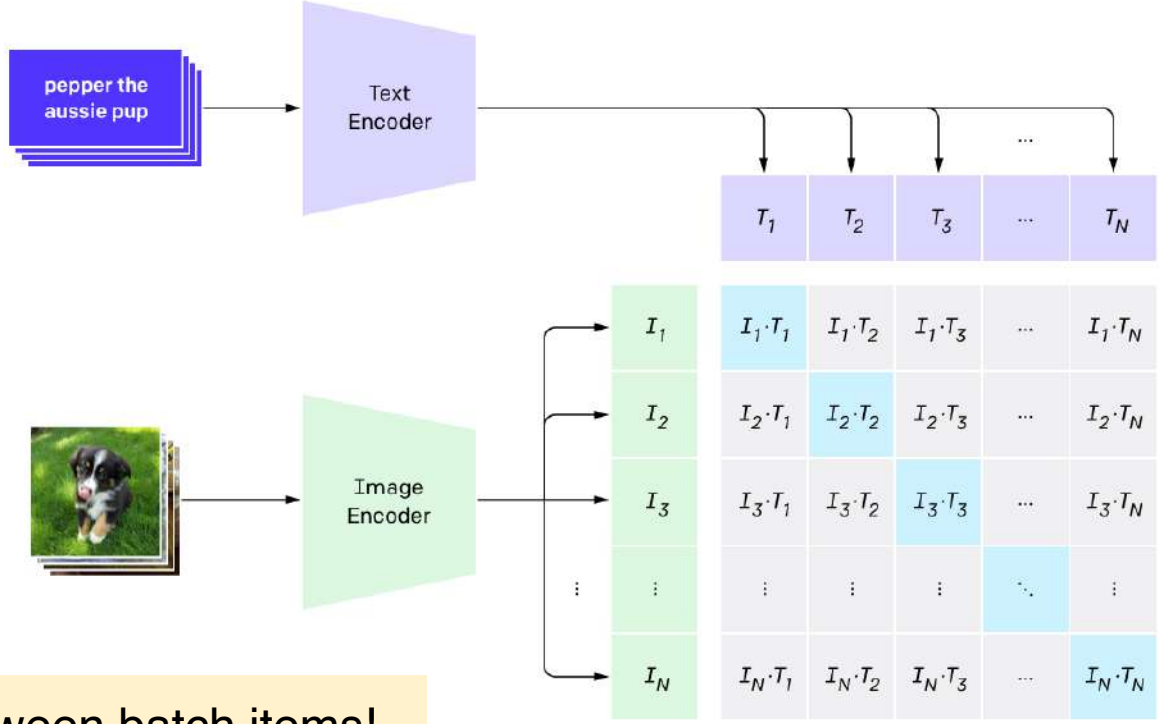
Ideally, this is our semantic distance metric.



CLIP Training

We get a bunch of paired text-image data

& train the text and image encoders to align its embeddings.



Loss is cross entropy between batch items!

https://openai.com/index/clip/

Contrastive Loss

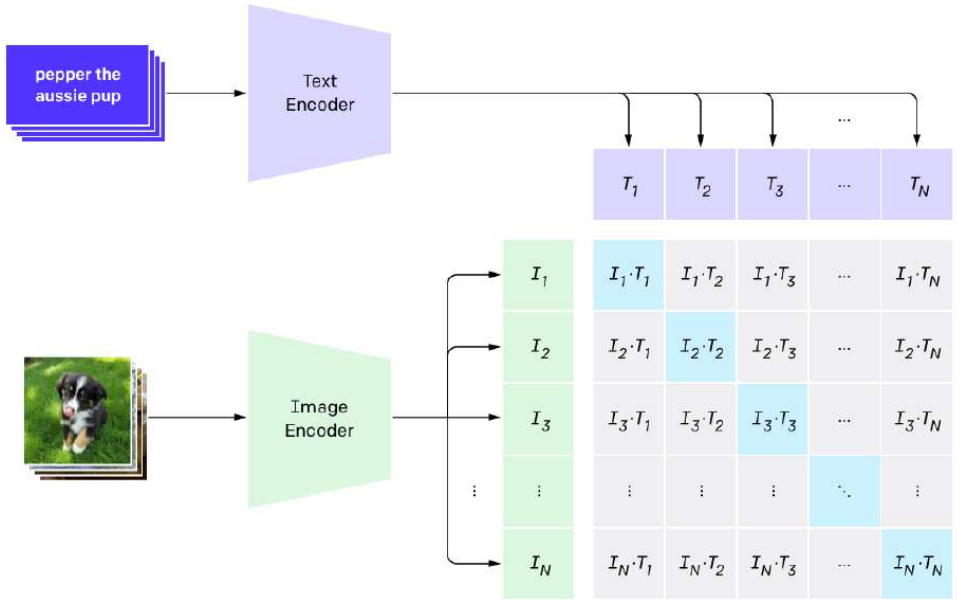
```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```



Contrastive Loss

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Image-to-Text Contrastive Loss:

$$L_{\text{image}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp(S_{ii})}{\sum_{j=1}^{N}\exp(S_{ij})}$$

Text-to-Image Contrastive Loss:

$$L_{\text{text}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp(S_{ii})}{\sum_{j=1}^{N}\exp(S_{ji})}$$

Final CLIP Loss:

$$L_{\text{CLIP}} = \frac{1}{2}(L_{\text{image}} + L_{\text{text}})$$

Real CLIP Training

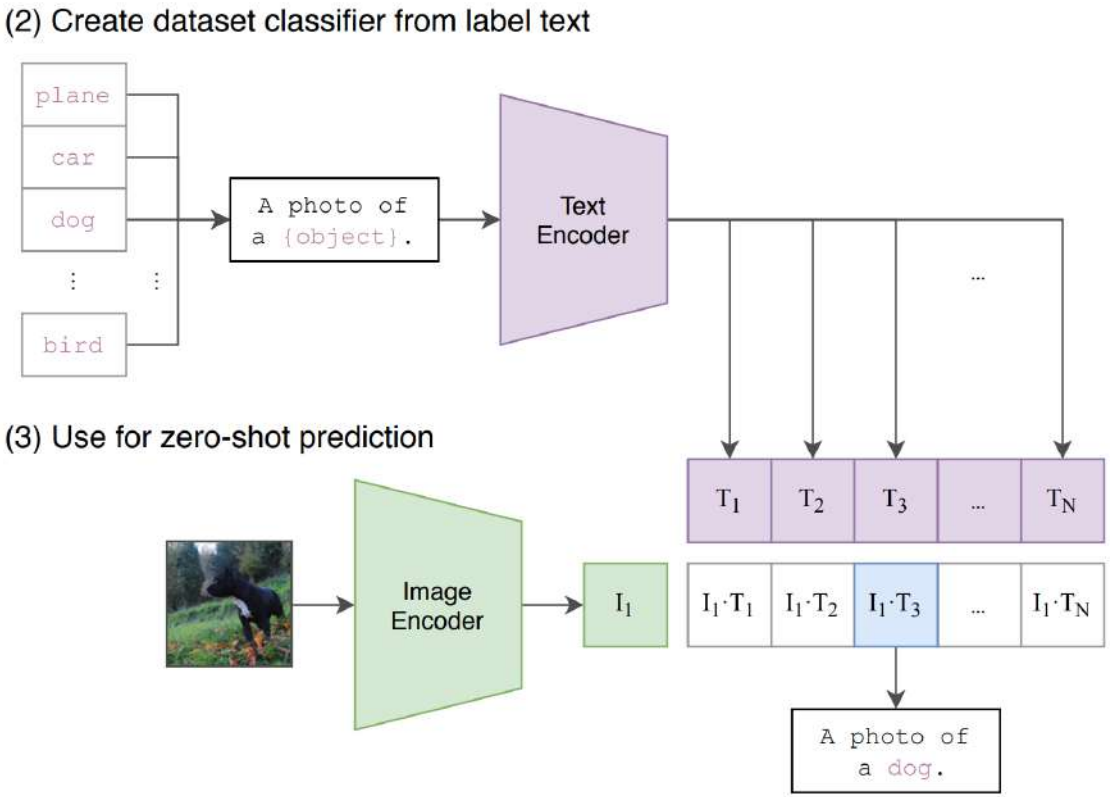
- 32 epochs of training
- Mini-batch size of **32,768** (not so mini)
- Hyperparameters were tuned with grid search, random search, and manual tuning
- Adam optimizer
- Largest ViT model took 12 days to train on 256 V100 GPUs

https://arxiv.org/pdf/2103.00020

Downstream CLIP Uses

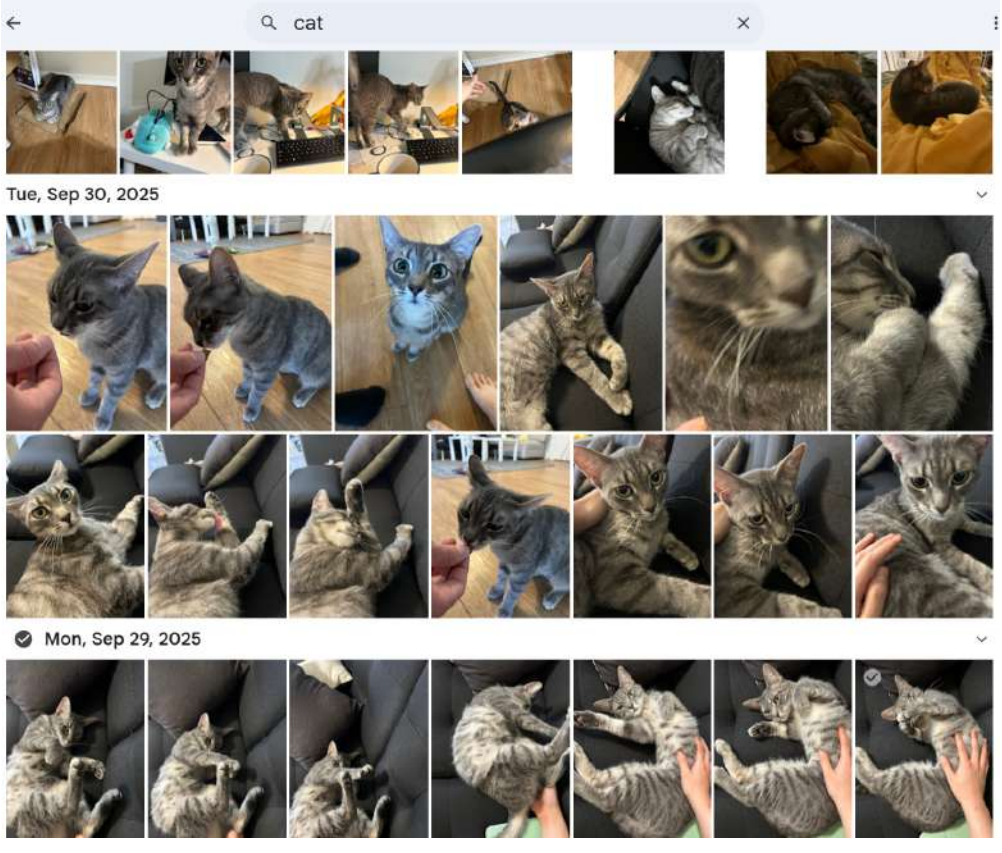
Zero-shot classification

(closed world)



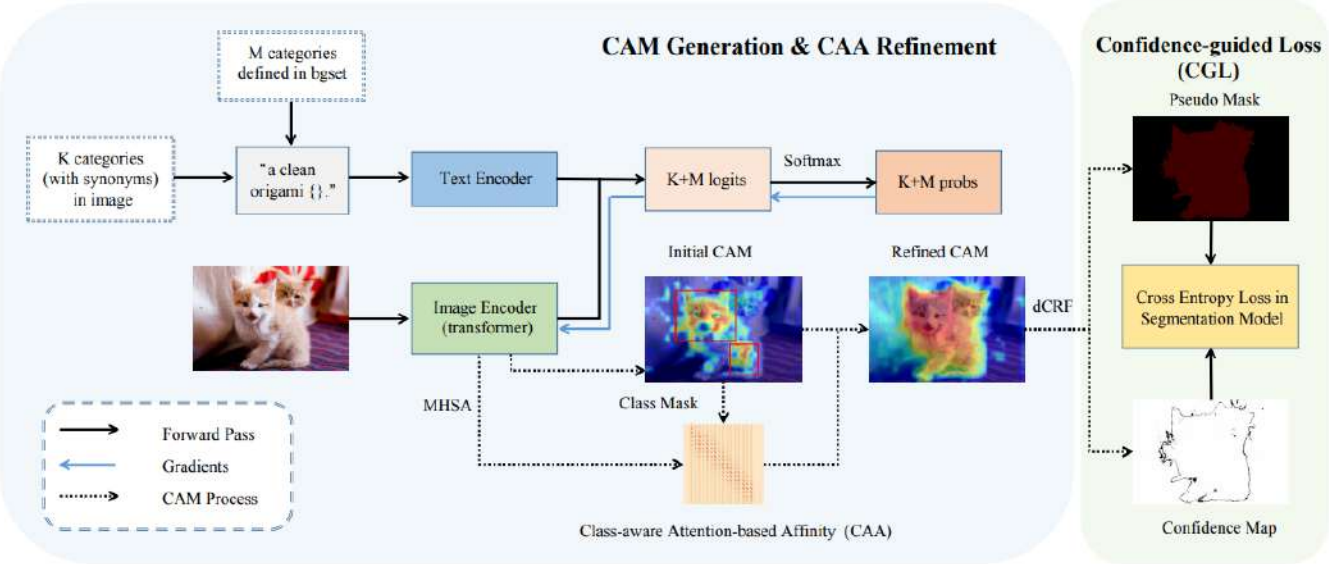
Downstream CLIP Uses

Visual search



Downstream CLIP Uses

Image Segmentation

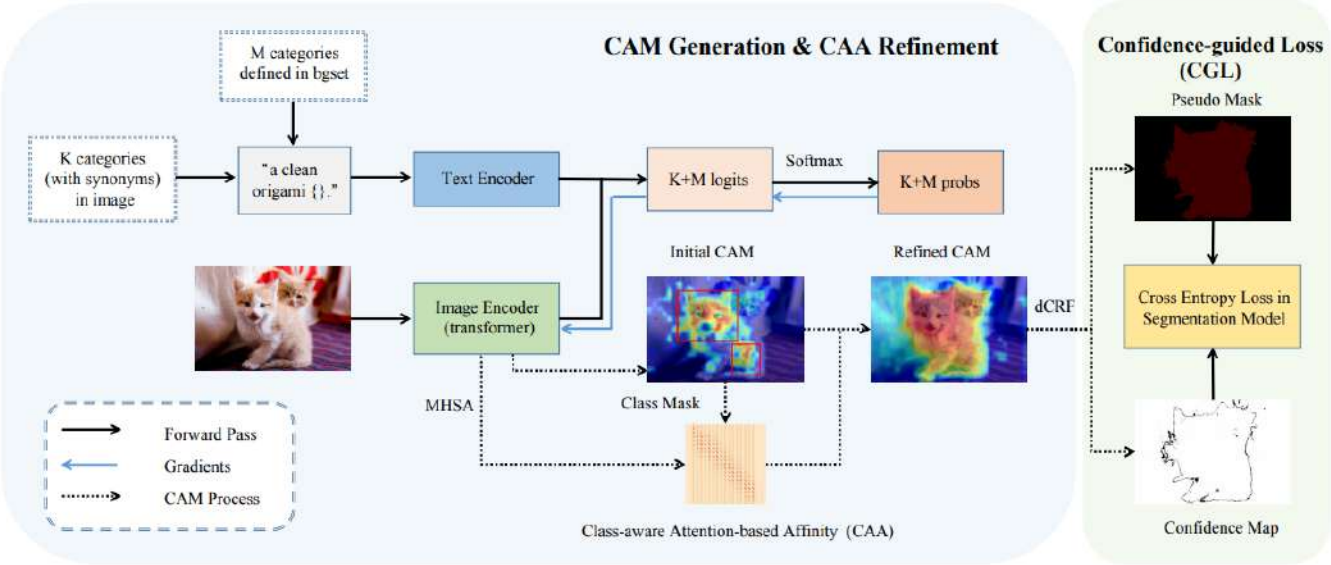


https://arxiv.org/pdf/2212.09506

Downstream CLIP Uses

Image Segmentation

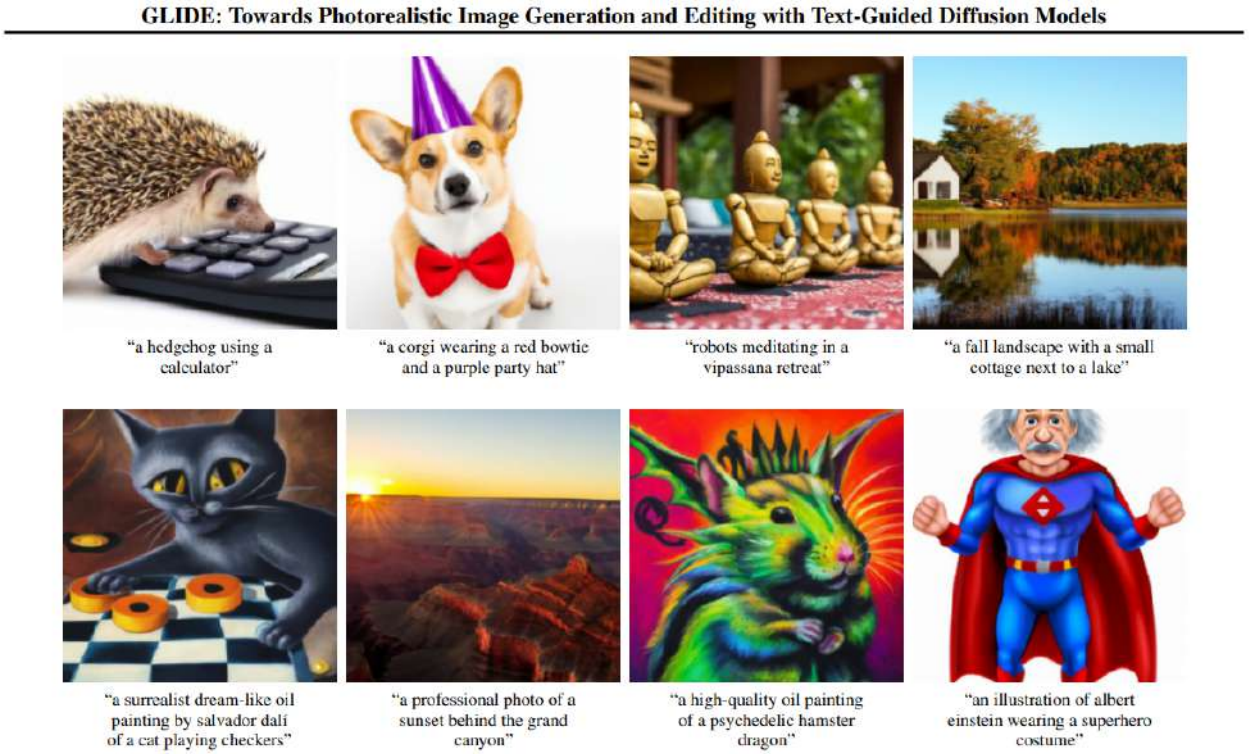
High level intuition:
If you identify the image regions that matter the most for score on “dog”, then that will be a segmentation map.



https://arxiv.org/pdf/2212.09506

Downstream CLIP Uses

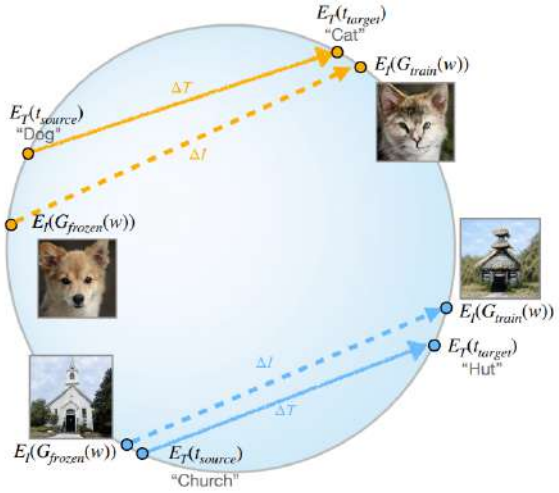
Diffusion guidance



https://arxiv.org/pdf/2112.10741

Downstream CLIP Uses

Style transfer



https://arxiv.org/pdf/2108.00946

Downstream CLIP Uses

Vision-language modeling

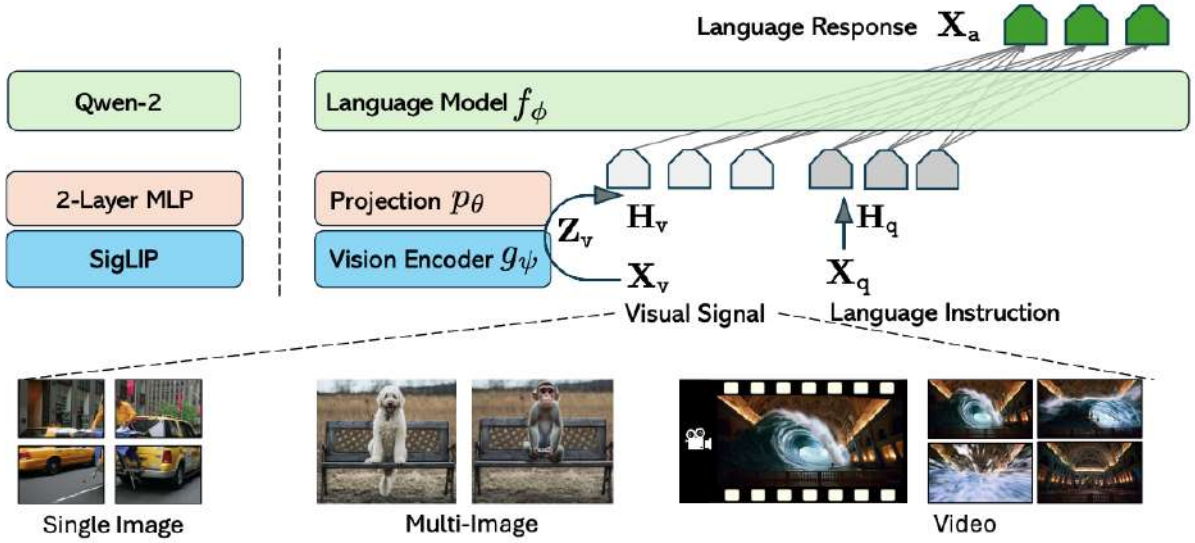


Figure 1: LLaVA-OneVision network architecture. Left: The current model instantiation; Right: the general form of LLaVA architecture in [83], but is extended to support more visual signals.

https://arxiv.org/abs/2408.03326

Downstream CLIP Uses

Vision-language modeling

Here, you leverage the aligned vision and text embeddings to do linguistic reasoning on both modalities

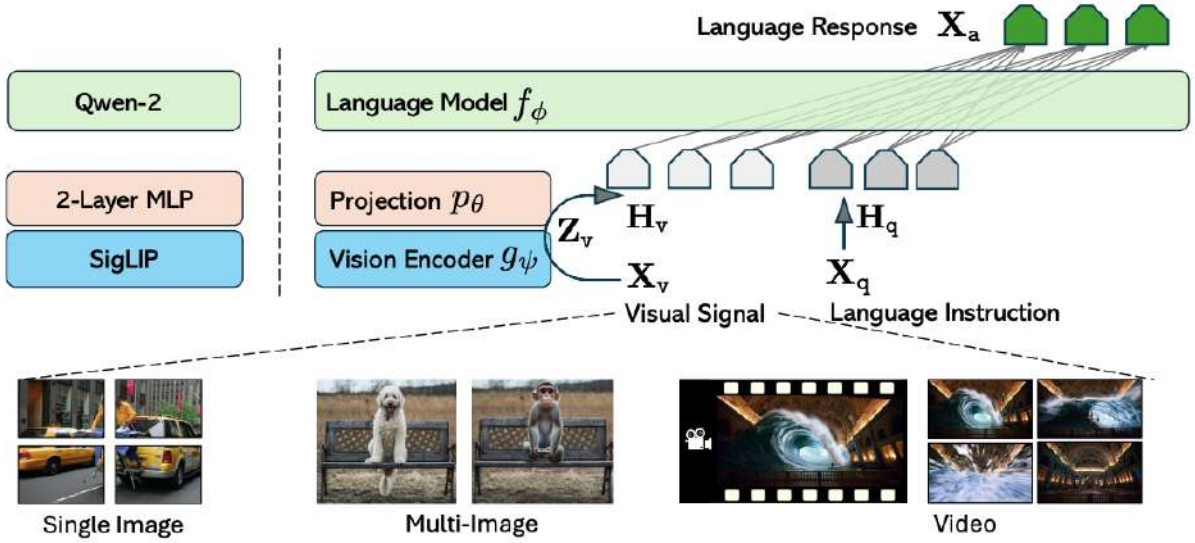


Figure 1: LLaVA-OneVision network architecture. Left: The current model instantiation; Right: the general form of LLaVA architecture in [83], but is extended to support more visual signals.

https://arxiv.org/abs/2408.03326

CLIP Limitations

Geometric limitations arise when compacting an image of a complex scene or a long sentence into single N-dimensional vectors!

Is CLIP ideal? No. Can we fix it? Yes!

Raphi Kang Yue Song Gerogia Gkioxari Pietro Perona
California Institute of Technology
Pasadena, CA
rkang@caltech.edu

https://arxiv.org/pdf/2503.08723

CLIP Limitations

Geometric limitations arise when compacting an image of a complex scene or a long sentence into single N-dimensional vectors!

If we have time: Ideal CLIP slide deck

https://arxiv.org/pdf/2503.08723

DINO (quick)

DINO is a *vision-only foundation model*.

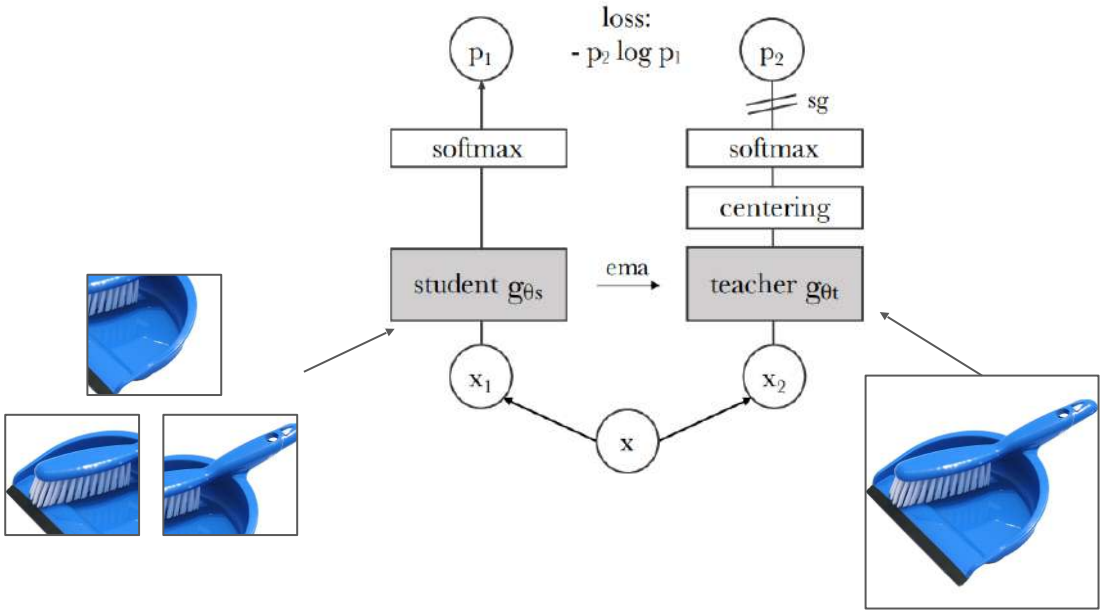
We saw earlier that CLIP can do text-guided image segmentation.
One of the great uses of DINO is that distinct attention heads “pay attention” to class-specific image features with *unsupervised training*.

https://arxiv.org/pdf/2104.14294

DINO (quick)

Unsupervised means no text labels at all.

DINO simply uses different image crops and learns to associate the global views to the local ones.

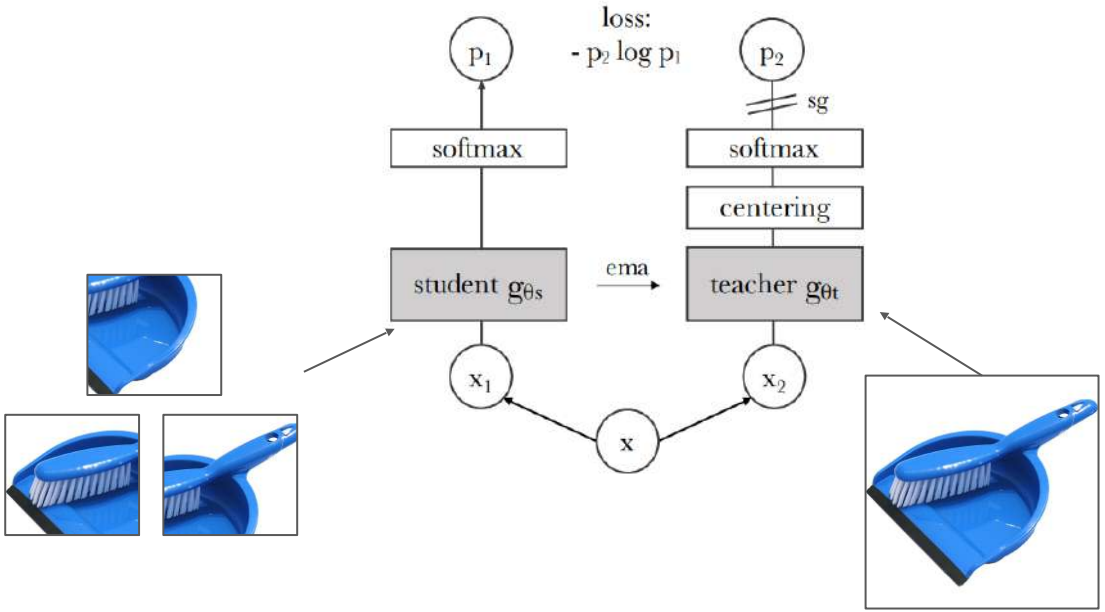


https://arxiv.org/pdf/2104.14294

DINO (quick)

DINO uses a *knowledge distillation* paradigm.

- “Student network” gets local views + gradient updates
- “Teacher network” gets global image views, but its weights only get updated as an EMA of the student’s weights



https://arxiv.org/pdf/2104.14294

Next Project Checkpoint

- We will be “standing on the shoulders of giants”
  - I.e., using *foundation models*
- I recommend you check out CLIP or DINO towards your final projects!

The End!

Questions?